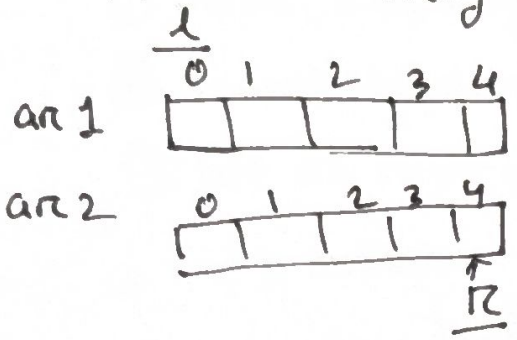
b. Two sum Revisited:

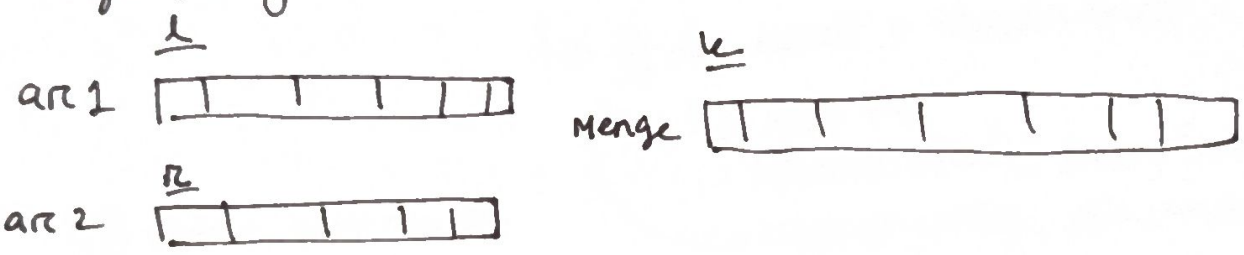
since both array is sorted in ascending order, ~~then~~ have 2 pointers.



sum = a[l] + a[r];
if sum > target: r--
elif sum < target: l++
else: return l, r

keep tracks of the best answer.

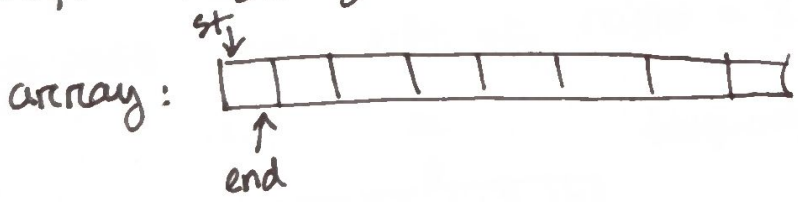
c. Merge Array:



repeat:
if arr1[l] < arr2[r] : Merge[k] = arr1[l]; l++;
: Merge[k] = arr2[r]; r++;
else:
K++;

d. longest Subarray Sum: [only for positive values]

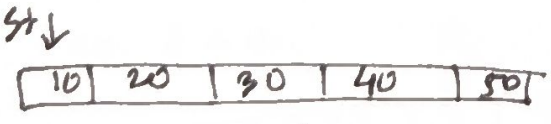
Implement sliding window ~~te~~ technique.



start with one element subarray.
curr-sum = ~~arr[st]~~ 0;

~~if a[st]~~

now, end pointer will increase from 0 → length. and each time add curr-sum += arr[end]. At some point;



; target = 70; if end increases again, it goes beyond 70, so start value has to change to minimize curr-sum.

loop (end → 0 → length → ++):
curr-sum += a[end]
while (curr-sum > target):
curr-sum -= a[st]
st ++;
update best answer

E. longest K-distinct subarray:

(i) Track frequency of element,

(ii) distinct counter = 0

(iii) left = 0, right = 0

↳ increase right

↳ if add frequency in hashtable

↳ if already added in table ; distinct counter unchanged

↳ else dis_count ++;

↳ if dis-count > target_count

↳ left ++

remove a[left] from frequency

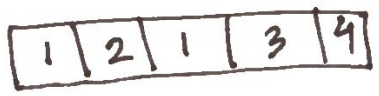
repeat until dis_count <= target count

(iv) update best result

Sample: [1, 2, 1, 2, 3] k = 2

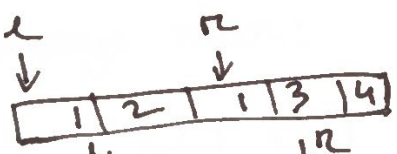
Ans: [1, 2, 1, 2] ⇒ length 4

at a given point where



, K = 2

in a case where left = 0, right = 2:



frequency {1:2, 2:1}

distinct = 2

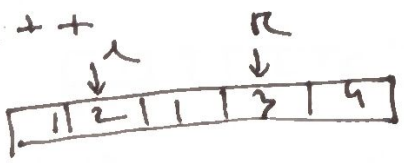
now r++ ⇒



frequency {1:2, 2:1, 3:1}

distinct = 3 [> k]

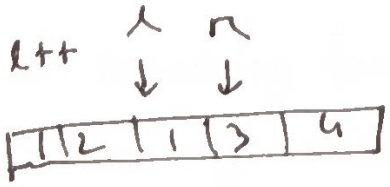
so, l++



freq: {1:1, 2:1, 3:1}

d = 3

again, l++



freq {1:1, 3:1}

d = 2

carry on, update the best result.

F. count the numbers:

- given the starting and ending values $[x, y]$
- search in $O(\log_2(n))$ time for first occurance of x
- " " " " " last " " y
- return the range in between.

G. Can you split the array?

lets say $\underline{3}$ subarrays of continuous array such that the max sum is lowest: $\overleftarrow{k}$



; 3 subarray.

~~subset~~ one way can be:  ⇒ max sum = 7

another :  ⇒ max sum = 5 ✓

" :  ⇒ max sum = 5. ✓

To solve this efficiently, instead of bruteforcing, we can use estimated guess to understand the range of solution.

(i) The lowest max-sum of an subarray will always be <u>one length</u> subarrays max:



: max = 5, ✓

so, lowest : max(arr)

(ii) highest value is the whole array as subarray [since positive]

 ⇒ max = 11

highest = sum(arr)

Now, instead of searching for answer, we search for the minimum possible valid sub arrays with the lowest max-sum

low = max(Arr)
high = sum(Arr)
mid = (low + high) // 2

if low doesn't exceed high:

check-valid-solution(mid) ⇒

↳ if true: means search for even smaller sum
↳ high = mid

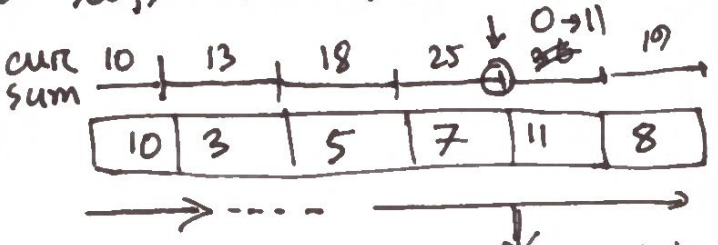
↳ else: no valid splitting found for target max-sum mid,
need to increase the threshold
↳ low = mid

loop

store best answer

check valid solution()

lets say, mid = 28; since, low = 11, high = 44



upto this point where cur-sum is less than mid, that is the first part/subarray. reset the cur-sum after. after each reset, add a counter ++. [initialized as 1]

now, if the counter <= K [number of splits] ⇒ return true
else : false.

H. Can you cut the rope:

Same as G, lowest length to cut = 1

highest " " = max(arr)
mid = (low + high)//2
check-valid(mid) → true: low = m // try bigger length
false: high = m // find smaller "

I. Triple trouble: [3sum problem]

① sort
② 3 pointer: Fixed : i, left : i+1, right : length-1, close-sum = ∞

l < r → i++

sum = a[i] + a[l] + a[r]
sum == 0 : done
< 0 : sum < close-sum: update close-sum; l++
else: r--

J. Searching is fun:

$x=3$, $R=7$, means # from 1 to N, the 7th number that is not divisible by 3.

If let's say total number range $1 \to 10$.

valid ones: 1, 2, 4, 5, 7, 8, 10 [7] <

invalid : 3, 6, 9. [3]

mathematically : valid range = $N - \lfloor N/x \rfloor$

$= 10 - 3 = 7$.

So, set a low value [1], high value [max int], mid = $(l+h)/2$, with the help of variable x, check the valid-range. if valid-range $< k$; expand search area [$l = m+1$], else [$h = m-1$]